1. System Overview

What is SmartRFP?

- Automates RFP response generation using AI pipeline
- Extracts requirements → Retrieves context from knowledge base → Generates proposal sections
- All generated content reviewed and approved by humans before export
- Reduces RFP response time from 20 hours to 3 hours

2. System Architecture

Core Pipeline

- **Extraction:** Regex + LLM to parse RFP and identify vendor requirements
- **Retrieval (RAG):** FAISS vector DB with semantic search to fetch relevant knowledge base documents
- **Pricing:** Tavily web search + internal rate card for cost estimation
- **Generation:** Groq LLM generates proposal sections with retrieved context
- **Evaluation:** RAGAS metrics (faithfulness, relevance, precision, recall) to measure quality

3. Technology Stack

Component	Technology	Why This Choice
LLM Provider	Groq (Llama 3.1)	10-50x faster than OpenAI, 10x cheaper, excellent quality for text generation
Vector Database	FAISS (now)	Lightweight, fast. Migrating to Pinecone for scalability
Web Search	Tavily API	Real-time market pricing data, lightweight alternative to full web crawling
Backend	FastAPI	Async, fast, production-ready. Alternative: Django (slower, heavier) or Flask (not async)
Frontend	Streamlit	Pure Python, no HTML/CSS needed. Rapid dev. Alternative: React (requires JS/TS, slov
Framework	Langchain	Unified interface for LLMs, prompts, chains. Simplifies Groq + FAISS integration
Observability	LangSmith	Traces every LLM call, shows latency/tokens/cost. Debugging RAG issues in real-time
Database	PostgreSQL	Scalable, reliable, JSONB for metadata. Replaces SQLite for production
Deployment	Docker	Containerized, reproducible. Easy scaling and multi-env (dev/staging/prod)

4. Technology Justification

Groq vs OpenAI vs Claude

- Groq: 50ms response time (OpenAI: 2-3 sec). Cost: $0.10/1M tokens (OpenAI: $15/1M). Winner for proposal generation.
- OpenAI: Better for complex reasoning, vision tasks. Not needed for our use case.
- Claude: Excellent but expensive and slower. Overkill for structured generation.

Streamlit vs React vs Vue

- Streamlit: Write UI in Python, deploys in seconds. Perfect for AI apps. No JS/CSS/HTML needed.
- React: Requires TypeScript, build pipeline, separate backend. 3x slower development.

- Vue: Similar complexity to React. Not worth it for this project scope.

FastAPI vs Django vs Flask

- FastAPI: Async from ground up, automatic OpenAPI docs, type hints. Fastest response time.
- Django: Full-featured but slower, synchronous by default. Overkill for API-only backend.
- Flask: Lightweight but not async. Would need async extensions, defeats the purpose.

FAISS → Pinecone Migration

- FAISS now: In-memory vector DB, fast, zero ops overhead. Good for <10M documents.
- Pinecone future: Managed service, serverless scaling. When knowledge base grows >100M documents.

5. Backend How It Works

- **Upload:** User uploads RFP (PDF/DOCX) → FastAPI receives file
- **Extract:** Regex + Groq LLM parse RFP → structured requirements list (SQL stored)
- **Retrieve:** For each requirement, FAISS finds top-3 similar docs from knowledge base (embeddings via Langchain)
- **Price:** Tavily searches market rates + internal rate card → cost estimate
- **Generate:** Groq generates proposal section with retrieved context (traced via LangSmith)
- **Evaluate:** RAGAS computes faithfulness, relevance, precision, recall scores
- **Save:** SQLAlchemy persists to PostgreSQL (requirements, sections, scores, audit log)
- **Review:** User sees dashboard, edits sections, approves before export

6. Observability

- **LangSmith:** Every Groq call traced. Shows prompt, tokens, latency, output. Real-time debugging.
- **Logging:** Each pipeline step logs input/output/execution time. Errors with full traceback.
- **Metrics:** Extraction accuracy, RAG precision, RAGAS scores, API costs, total pipeline time.
- **Audit Trail:** Every action logged (upload, generate, edit, approve, export) for compliance.

7. Safety & Guardrails

- **Grounding:** LLM instructed to only use retrieved context. No hallucination without source.
- **Faithfulness Scoring:** RAGAS metric checks if facts in output are supported by context.
- **Human Review Gate:** All output requires explicit human approval before export. No auto-export.
- **Privacy:** Knowledge base never sent to external APIs. Only anonymized requirements sent to Groq.
- **Validation:** File size limit (50MB), only PDF/DOCX/TXT accepted. Input sanitization before LLM.

8. Evaluation Metrics (RAGAS)

Metric	What It Measures	Target	Current
Faithfulness	Generated text supported by context (no hallucination)	>0.80	0.82
Answer Relevance	Generated section answers the requirement	>0.75	0.79
Context Precision	Retrieved chunks are actually useful	>0.70	0.74
Context Recall	All necessary info was retrieved	>0.70	0.76

9. Test Coverage

- **Extraction Tests:** Verify regex + LLM correctly identify RFP requirements. 87% accuracy on test set.
- **RAG Tests:** FAISS retrieval returns relevant docs. Top-3 similarity >0.7. Low-relevance queries return nothing.
- **Generation Tests:** Groq output is relevant, coherent, professional. RAGAS scores validated.
- **Integration Tests:** Full pipeline end-to-end. PostgreSQL persistence, audit logs, export formats.
- **Error Handling:** PDF parsing failures, Groq API down, FAISS unavailable → graceful fallback.

10. Docker Deployment

- **Containerization:** Dockerfile for Streamlit frontend + FastAPI backend. PostgreSQL in separate container.
- **Docker Compose:** Single 'docker-compose up' spins up: Streamlit, FastAPI, PostgreSQL, Redis (cache).
- **Environment Variables:** Groq API key, Tavily key, LangSmith key in .env file (not hardcoded).
- **Scaling:** Kubernetes-ready. StatelessFastAPI can scale horizontally. PostgreSQL managed separately (RDS/Cloud SQL).
- **CI/CD:** GitHub Actions: lint, test, build Docker image, push to registry (Docker Hub/ECR).

11. Market Opportunity

- **Problem:** RFP response costs $2,250-6,000 per proposal. Mid-market firms respond 20-50/year = $45K-300K annually.
- **TAM:** ~8,000 IT services + 15,000 consulting + 50,000 software vendors = $3.7B addressable market.
- **Solution Cost:** $28/RFP (API + infra). 30 RFPs/year = $840/month vs $3,000/RFP manual cost.
- **ROI:** $90K/year (manual) → $23K/year (with SmartRFP) = $66K savings. 6.6x ROI. Payback: 1.8 months.

12. Product Roadmap

- **Phase 1 (Now):** Core pipeline (extract, RAG, generate, evaluate), Streamlit UI, PostgreSQL, LangSmith tracing.
- **Phase 2 (Q2):** Pinecone migration, multi-turn editing, template customization, email integration.
- **Phase 3 (Q3):** Win/loss analysis, pricing ML model, auto-indexing (Confluence/Drive), multi-language.
- **Phase 4 (Q4):** REST API, SSO (SAML), advanced analytics dashboard, white-label option.

13. Why SmartRFP

- **vs ChatGPT:** No RFP workflow. Hallucination risk. No audit trail.
- **vs Proposify:** Template-based only. No AI generation. Slow.
- **vs HubSpot:** Minimal AI. Not RFP-specific. Heavy and expensive.
- **SmartRFP Edge:** Purpose-built for RFPs. Grounded in company knowledge. RAGAS quality scores. Human review gate.